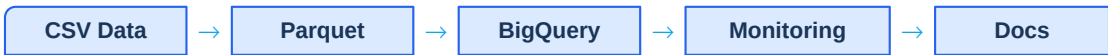


Data Engineering Project

Team Work Distribution & Collaboration Plan

Project Overview

This document outlines the structured work distribution for a team of five engineers building an end-to-end ETL pipeline. Raw CSV data is collected, cleaned, and converted to Parquet format, then loaded into Google BigQuery for analytics. The project is divided into five milestones, each owned by one team member, with clear hand-off points and shared integration responsibilities.



Team Assignment Summary

Person	Milestone	Main Role	Key Tools
Person 1	Milestone 1	Data Collection & Preprocessing Lead	Python, Polars, Pandas, Parquet, DuckDB
Person 2	Milestone 2	Transformation & Validation Engineer	Polars, Pandas, Great Expectations, dbt
Person 3	Milestone 3	Deployment & Pipeline Engineer	Airflow, BigQuery, Docker, GCP
Person 4	Milestone 4	MLOps, Monitoring & Automation Lead	Airflow, Cloud Logging, AlertManager, Grafana
Person 5	Milestone 5	Documentation & Presentation Lead	Markdown, Notion, Jupyter, Slides

Detailed Role Descriptions

Person 1	Data Collection & Preprocessing Lead
----------	--------------------------------------

Milestone 1 — Data Collection, Exploration & Preprocessing

Key Tasks:

- Identify and download open datasets (weather, transportation, or similar CSV sources).
- Perform exploratory data analysis (EDA): inspect schemas, data types, null values, and duplicates.
- Clean raw data: handle missing values, fix inconsistent formats, remove outliers.
- Convert cleaned CSV files to Parquet format using Polars or PyArrow for efficient storage.
- Document dataset sources, schema decisions, and cleaning steps in a shared wiki.
- Hand off validated Parquet files to Person 2 via shared storage (GCS bucket or local mount).

Tools & Technologies:	Python 3.11, Polars, Pandas, PyArrow, DuckDB, Jupyter Notebook, Google Cloud Storage
-----------------------	--

Person 2

Transformation & Validation Engineer

Milestone 2 — Model / System Development & Transformation

Key Tasks:

- Design and implement data transformation workflows to apply business logic on Parquet data.
- Build reusable transformation modules using Polars (preferred) and Pandas where needed.
- Write data quality validation rules (schema checks, range checks, referential integrity).
- Integrate Great Expectations or dbt tests for automated validation assertions.
- Produce a data quality report summarising pass/fail metrics for each transformation step.
- Deliver clean, validated Parquet files ready for BigQuery ingestion to Person 3.

Tools & Technologies: Python, Polars, Pandas, Great Expectations, dbt Core, PyArrow, Jupyter Notebook

Person 3

Deployment & Pipeline Engineer

Milestone 3 — Deployment (Batch Pipeline to BigQuery)

Key Tasks:

- Design the BigQuery target schema: tables, partitioning strategy, and clustering keys.
- Write and test BigQuery load jobs to ingest validated Parquet files efficiently.
- Containerise the pipeline using Docker to ensure reproducible execution environments.
- Build the Airflow DAG (or Cloud Composer workflow) for scheduled batch execution.
- Configure GCP service accounts, IAM roles, and secure credential management.
- Test end-to-end pipeline execution (staging environment) and document runbooks.

Tools & Technologies: Apache Airflow, Google BigQuery, Docker, GCP (Cloud Storage, IAM), Python, bash

Person 4

Monitoring & Automation Lead

Milestone 4 — MLOps / Monitoring & Automation

Key Tasks:

- Implement pipeline health checks: row-count assertions, freshness checks, schema drift alerts.
- Set up GCP Cloud Logging and Cloud Monitoring dashboards for job status and latency.
- Configure alerting rules (email / Slack notifications) for pipeline failures or SLA breaches.
- Automate pipeline triggers using Airflow sensors or Cloud Scheduler cron jobs.
- Build a lightweight Grafana or Looker Studio dashboard for data freshness visibility.
- Write runbooks for on-call incident response and pipeline restarts.

Tools & Technologies: Apache Airflow, GCP Cloud Logging, Cloud Monitoring, Grafana, Looker Studio, PagerDuty / Slack, Python

Milestone 5 — Final Documentation, Demo & Presentation

Key Tasks:

- Consolidate all technical documentation: architecture diagram, ETL logic, BigQuery table schemas.
- Write a clear README and user guide for pipeline setup, configuration, and execution.
- Create a Jupyter Notebook demo that walks through sample queries on BigQuery results.
- Produce the final presentation deck covering project goals, design decisions, and outcomes.
- Record or run the live pipeline demo during the final presentation session.
- Compile lessons learned, future improvements, and a project retrospective document.

Tools & Technologies:

Markdown, Notion / Confluence, Jupyter Notebook, Google Slides / PowerPoint, Lucidchart (architecture diagram)

Team Collaboration & Integration Strategy

Shared Repository

All code lives in a single Git repository (GitHub/GitLab). Each person works in a dedicated feature branch; pull requests require at least one peer review before merging to main.

Data Hand-off Protocol

Person 1 deposits Parquet files into a shared GCS bucket. Person 2 reads from that bucket, applies transformations, and writes validated outputs back. Person 3 loads validated files into BigQuery. Each hand-off is gated by a data quality check.

Weekly Sync Meetings

A 30-minute stand-up every week covers blockers, integration issues, and upcoming hand-offs. A shared Notion board tracks milestone progress and open issues.

Shared Configuration

Environment variables, GCP credentials, and pipeline parameters are stored in a central .env file and a config.yaml. Docker Compose ensures everyone runs the same environment locally.

CI/CD Pipeline

GitHub Actions runs unit tests, data validation checks, and linting on every pull request, preventing broken code from reaching the main branch.

Documentation-as-Code

Person 5 maintains a living docs site (MkDocs or Notion). All team members are responsible for keeping their own section up to date with decisions and schema changes.

Each team member owns their milestone end-to-end and is accountable for delivering a working, tested, and documented artefact. Cross-milestone dependencies are surfaced early via the weekly sync so the team can resolve blockers before they impact downstream work.